Security and Compliance Framework for BankTech's LLM Financial Advisor

Module 2 Hands-On Project Report — Peer Review submission

Author: Luxna Ramnauth | Date: 3 October 2026

Before you submit: this report is built from the Module2_Security starter and solution files. Items marked **[FILL IN]** need your own runs (gateway test output, screenshots, ELK results, your own validator results). The injection-test table in Requirement 3 is the one measured result included: I ran the *course reference solution* against the provided tests. Replace it with results from your own validator before you submit.

Executive summary

BankTech's penetration test found unsecured API endpoints, prompt-injection susceptibility and inadequate audit logging ahead of a launch to 5 million customers. This framework closes those gaps in layers: JWT authentication and least-privilege IAM at the gateway, tiered rate limiting (1,000 / 5,000 / 20,000 requests per hour), an input-validation and sanitization layer for prompts, a PII output filter, and tamper-evident audit logging with retention and GDPR deletion support, all mapped to a 20-control compliance matrix.

Vulnerability found	Control implemented	Requirement
Unsecured API endpoints	JWT auth at gateway; IAM least privilege; per-tier rate limits	1, 2
Prompt-injection susceptibility	Regex/keyword/structure validation; sanitization; length limit	3
PII leakage in responses	Regex PII detection, masking and response blocking	4
Inadequate audit logging	Structured hashed-prompt audit log, ELK, retention, RBAC, encryption	5

Requirement 1 — API authentication configuration

Kong validates a JWT on every request to the LLM service. The reference configuration verifies the `exp` and `iss` claims, accepts the token only from the Authorization header (not URL parameters or cookies), and caps token lifetime at 1 hour.

JWT plugin (Kong)

```
plugins:
  # ------------------------------------------------------------------------
  # SOLUTION: JWT Authentication - Properly configured
  # ------------------------------------------------------------------------
  - name: jwt
    service: llm-inference-service
    config:
      uri_param_names: []
      cookie_names: []
      header_names:
        - Authorization
      claims_to_verify:
        - exp
        - iss
      key_claim_name: iss
      secret_is_base64: false
      run_on_preflight: true
      maximum_expiration: 3600  # 1 hour max token lifetime
```

IAM least-privilege design

Role	Allowed	Explicitly not allowed
Inference server role	S3 read of the model-weights bucket; CloudWatch Logs write	Any write to S3; IAM changes; wildcard resources
Application role	Invoke the LLM endpoint through the gateway	Direct access to model storage or the inference host

IAM policy JSON documents: **[FILL IN]** (attach the two policies you wrote).

Test evidence

```
# no token -> expect 401
curl -i https://$GATEWAY/v1/inference -d '{"prompt":"hello"}'
# expired / bad-signature / wrong-issuer token -> expect 401
curl -i -H "Authorization: Bearer $BAD_TOKEN" https://$GATEWAY/v1/inference ...
# valid token -> expect 200
curl -i -H "Authorization: Bearer $GOOD_TOKEN" https://$GATEWAY/v1/inference ...
```

Test	Expected	Observed
No token	401	**[FILL IN]**
Expired token	401	**[FILL IN]**
Bad signature	401	**[FILL IN]**
Wrong issuer	401	**[FILL IN]**
Valid token	200	**[FILL IN]**

Requirement 2 — Rate limiting implementation

Each consumer tier gets its own Kong `rate-limiting` plugin, backed by Redis so limits hold across gateway instances. Per-minute caps act as the burst limit; quota headers are returned to clients (`hide_client_headers: false`), and over-limit calls get HTTP 429.

Tier	Per hour	Per minute (burst)
Basic	1,000	100
Professional	5,000	500
Enterprise	20,000	2,000

```
# SOLUTION: Rate Limiting - Basic Tier (1,000 requests/hour)
# ------------------------------------------------------------------------------
- name: rate-limiting
  consumer: basic-tier-client
  config:
    hour: 1000
    minute: 100
    policy: redis
    redis_host: ${REDIS_HOST}
    redis_port: 6379
    redis_password: ${REDIS_PASSWORD}
    redis_database: 0
    fault_tolerant: true
    hide_client_headers: false
    error_code: 429
    error_message: "Rate limit exceeded. Please upgrade your plan or try again later."
```

Evidence of 429 responses: **[FILL IN]** (loop of requests until the limit trips; paste status codes). Quota headers observed (e.g. `X-RateLimit-Remaining-hour`): **[FILL IN]**.

Note: `fault_tolerant: true` means that if Redis is unreachable the gateway lets traffic through unlimited. That favours availability; for a cost-attack threat model consider failing closed or adding a secondary local limit.

Requirement 3 — Prompt-injection defense

The reference `PromptValidator` layers five checks: regex patterns grouped by attack category (instruction override, jailbreak, prompt leaking, and others), keyword analysis, structural analysis, encoding detection, and a placeholder for semantic analysis. Results carry a threat level; `LLMSecurityPipeline` wires validation before the model and output filtering after it.

Pattern excerpt (instruction override and jailbreak categories)

```
    4. Encoding detection
    5. Semantic analysis (placeholder for ML-based)
    """

    # SOLUTION: Comprehensive injection patterns
    INJECTION_PATTERNS = {
        AttackCategory.INSTRUCTION_OVERRIDE: [
            r"(?i)ignore\s+(all\s+)?(previous|above|prior)\s+(instructions?|rules?|guidelines?)",
            r"(?i)disregard\s+(all\s+)?(previous|above|prior)\s+(instructions?|rules?)",
            r"(?i)forget\s+(everything|all)\s+(you\s+)?(know|learned|were\s+told)",
            r"(?i)new\s+instructions?:\s*",
            r"(?i)override\s+(system|safety|security)\s+(prompt|instructions?)",
            r"(?i)you\s+are\s+now\s+(a|an)\s+",
            r"(?i)from\s+now\s+on,?\s+(you|ignore|forget)",
            r"(?i)stop\s+being\s+(a|an)\s+",
            r"(?i)act\s+as\s+if\s+(you\s+)?(have\s+)?no\s+restrictions?",
            r"(?i)\[system\]|\[admin\]|\[developer\]",
        ],
        AttackCategory.JAILBREAK: [
            r"(?i)dan\s*mode",
            r"(?i)developer\s+mode",
            r"(?i)jailbreak",
            r"(?i)bypass\s+(safety|security|content)\s+(filter|restrictions?)",
            r"(?i)pretend\s+(you\s+)?(don'?t|do\s+not)\s+have\s+(any\s+)?restrictions?",
```

Sanitization and limits

- Strip control characters and escape sequences; normalise whitespace.
- Enforce a maximum prompt length (the reference class defaults to 10,000 characters; the course target is 4,096 tokens — set whichever your design requires).
- Keep the system prompt separate from user text so it cannot be overridden by content in the user turn.
- Log blocked requests as security events with a hash and category, not the malicious text.

Test results: reference solution against M2_injection_tests.json

I ran the course reference `PromptValidator` (default settings) against all 18 provided cases. It blocked **4 of 14** attack cases and correctly allowed **4 of 4** legitimate requests. This is not your implementation's result; replace it with your own run.

ID	Category	Case	Expected	Reference result
DI-001	direct injection	Ignore instructions attack	BLOCKED	**BLOCKED**
DI-002	direct injection	Role switching attack	BLOCKED	**ALLOWED**
DI-003	direct injection	System prompt extraction	BLOCKED	**ALLOWED**
DI-004	direct injection	Developer mode simulation	BLOCKED	**BLOCKED**

ID	Category	Case	Expected	Reference result
DI-005	direct injection	Hypothetical scenario bypass	BLOCKED	**ALLOWED**
II-001	indirect injection	Hidden instruction in document	BLOCKED	**ALLOWED**
II-002	indirect injection	URL with malicious content	BLOCKED	**ALLOWED**
II-003	indirect injection	Base64 encoded payload	BLOCKED	**BLOCKED**
PII-001	pii extraction	Direct PII request	BLOCKED	**ALLOWED**
PII-002	pii extraction	Indirect PII extraction	BLOCKED	**ALLOWED**
PII-003	pii extraction	Credit card request	BLOCKED	**ALLOWED**
PII-004	pii extraction	Password retrieval attempt	BLOCKED	**ALLOWED**
SQL-001	sql nosql injection	SQL injection in query	BLOCKED	**BLOCKED**
SQL-002	sql nosql injection	NoSQL injection	BLOCKED	**ALLOWED**
LEG-001	legitimate requests	Balance inquiry	ALLOWED	**ALLOWED**
LEG-002	legitimate requests	Transaction history	ALLOWED	**ALLOWED**
LEG-003	legitimate requests	General question	ALLOWED	**ALLOWED**
LEG-004	legitimate requests	Help request	ALLOWED	**ALLOWED**

Finding: pattern-based detection alone missed role-switch, prompt-extraction, hypothetical-bypass, hidden-document and most PII-extraction cases in the provided suite. This supports the course tip that no single control is sufficient. To close the gap, extend the patterns for the missed categories, add semantic or model-based classification (for example NeMo Guardrails), treat retrieved documents and URLs as untrusted, keep the model's tool permissions minimal, and rely on output filtering and gateway authorization as independent layers. After each change, re-run the full suite and also confirm the legitimate cases still pass.

Your own results (after your changes): attacks blocked **[FILL IN]** / 14; legitimate allowed **[FILL IN]** / 4.

Security-event logging configuration: **[FILL IN]** (show a blocked-request log entry: event type, threat level, prompt hash; no raw prompt).

Requirement 4 — PII output filtering

`OutputFilter` scans each model response with compiled regex patterns, replaces matches with fixed placeholders, assigns a risk score (SSN, credit card, API keys and JWTs highest) and records detections without storing the matched values. A content classifier can then block the whole response; the pipeline returns a generic refusal in that case.

PII patterns and redaction rules (excerpt)

```
PII_PATTERNS = {
    "ssn": (r'\b\d{3}-\d{2}-\d{4}\b', "XXX-XX-XXXX"),
    "ssn_no_dash": (r'\b\d{9}\b', "XXXXXXXXX"),
    "credit_card": (r'\b\d{4}[-\s]?\d{4}[-\s]?\d{4}[-\s]?\d{4}\b', "XXXX-XXXX-XXXX-XXXX"),
    "email": (r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', "[EMAIL REDACTED]"),
    "phone_us": (r'\b(\+1[-.\s]?)?\(?\d{3}\)?[-.\s]?\d{3}[-.\s]?\d{4}\b', "[PHONE REDACTED]"),
    "phone_intl": (r'\b\+\d{1,3}[-.\s]?\d{1,14}\b', "[PHONE REDACTED]"),
    "ip_address": (r'\b\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\b', "[IP REDACTED]"),
    "account_number": (r'\b[A-Z]{2}\d{2}[A-Z0-9]{4}\d{7}([A-Z0-9]?){0,16}\b', "[ACCOUNT REDACTED]"),
    "date_of_birth": (r'\b(0[1-9]|1[0-2])/(0[1-9]|[12]\d|3[01])/((19|20)\d{2})\b', "[DOB REDACTED]"),
    "passport": (r'\b[A-Z]{1,2}\d{6,9}\b', "[PASSPORT REDACTED]"),
    "driver_license": (r'\b[A-Z]\d{7,8}\b', "[DL REDACTED]"),
    "api_key": (r'\b(sk|pk|api|key)[-_]?[A-Za-z0-9]{20,}\b', "[API_KEY REDACTED]"),
    "jwt_token": (r'\beyJ[A-Za-z0-9-_]+\.eyJ[A-Za-z0-9-_]+\.[A-Za-z0-9-_]+\b', "[JWT REDACTED]"),
}
```

Data type	Masked as	Risk weight
SSN / credit card	XXX-XX-XXXX / XXXX-XXXX-XXXX-XXXX	1.0
Bank account (IBAN-style)	[ACCOUNT REDACTED]	0.9
API key / JWT	[API_KEY REDACTED] / [JWT REDACTED]	0.95
Email / phone	[EMAIL REDACTED] / [PHONE REDACTED]	0.5

Detection and blocking tests / redaction examples: **[FILL IN]** (input response text with fake PII → masked output; include at least SSN, card, account number and balance).

Note: the 9-digit SSN pattern and phone patterns can match harmless numbers (order IDs, timestamps). Expect false positives and tune with real traffic samples; balances need a separate rule because the sample patterns cover identifiers rather than amounts.

Requirement 5 — Compliance-ready audit logging

`AuditEvent` is a structured record with request, user, API, LLM, security and compliance fields. Prompts and responses are stored as hashes and lengths, never as content. Each event carries a hash chained to the previous event so tampering is detectable.

Log schema (excerpt)

```
class AuditEvent:
    """
    SOLUTION: Comprehensive audit event with all compliance fields.
    """
    # Core fields
    event_id: str
    timestamp: str
    event_type: str
    severity: str

    # Request context
    request_id: Optional[str] = None
    correlation_id: Optional[str] = None
    session_id: Optional[str] = None

    # User context
    user_id: Optional[str] = None
    consumer_id: Optional[str] = None
    client_ip: Optional[str] = None
    user_agent: Optional[str] = None

    # API context
    endpoint: Optional[str] = None
    method: Optional[str] = None
    status_code: Optional[int] = None
    response_time_ms: Optional[float] = None

    # LLM-specific fields
    model_id: Optional[str] = None
    model_version: Optional[str] = None
    prompt_hash: Optional[str] = None
    prompt_length: Optional[int] = None
    response_hash: Optional[str] = None
    response_length: Optional[int] = None
    token_count_input: Optional[int] = None
    token_count_output: Optional[int] = None
    ...
    resource_id: Optional[str] = None
    action: Optional[str] = None
    outcome: Optional[str] = None
    error_code: Optional[str] = None
    error_message: Optional[str] = None
    metadata: Optional[Dict[str, Any]] = None

    # Integrity
    ...

    def _calculate_hash(self) -> str:
        """Calculate tamper-evident hash of the event."""
        hash_data = {
            "event_id": self.event_id,
            "timestamp": self.timestamp,
            "event_type": self.event_type,
            "user_id": self.user_id,
            "request_id": self.request_id,
            "previous_hash": self.previous_hash
        }
        hash_string = json.dumps(hash_data, sort_keys=True)
        return hashlib.sha256(hash_string.encode()).hexdigest()
```

Retention and compliance controls

Control	Implementation
Retention	Logger default retention_days = 2,555 (about 7 years). Course targets: HIPAA 7 years, SOC 2 6 years. The matrix itself requires a minimum of 1 year (SEC-011). Confirm the final period with your compliance team.
GDPR erasure	Pseudonymize user identifiers; keep hashed prompts only (matrix SEC-008 already marked Implemented); 30-day response requirement (SEC-007).
Access control	Role-based Kibana/Elasticsearch roles: auditors read-only; engineers no access to raw security index.
Encryption	TLS in transit; encryption at rest on Elasticsearch volumes / S3.
Separation	Security events and operational metrics in separate indices; alerts on anomalous access.
Integrity	Hash chain (previous_hash / event_hash); IP masking helper available for client addresses.

ELK configuration (docker-compose, Logstash pipeline, index lifecycle): **[FILL IN]**. Screenshot of Kibana with LLM audit events: **[FILL IN]**.

Compliance matrix summary

Measure	Value
Controls in matrix	20
By framework	SOC 2: 10, GDPR: 4, PCI-DSS: 3, OWASP LLM: 3
Risk level	Critical: 5, High: 11, Medium: 4
Status at start	TODO 19, Implemented 1
Status now (yours)	**[FILL IN]**

How this project's controls map to the matrix

Control IDs	Covered by
SEC-001, 002, 003	JWT + consumer tiers + rate limits (Req. 1, 2)
SEC-004, 014	PII output filtering (Req. 4)
SEC-012, 013	Prompt-injection and jailbreak defense (Req. 3)
SEC-008, 009, 010, 011	Audit logging, alerting, retention (Req. 5)
SEC-007	GDPR erasure via pseudonymization (Req. 5)
SEC-005, 015–20	TLS, image/dependency scanning, network isolation, IR plan, change management — outside this module's code; list the status honestly

Sample audit queries

```
# Failed authentications in the last 24h (SOC 2 access monitoring)
event_type:"authentication" AND outcome:"failure" AND @timestamp >= now-24h

# Blocked prompt-injection attempts by user (security review)
event_type:"security_event" AND security_flags:"prompt_injection" | stats count by user_id

# All activity for one data subject (GDPR access / erasure request)
user_id:"<pseudonymized-id>" AND @timestamp >= now-365d

# Responses where PII was detected and masked
pii_detected:true | stats count by pii_types
```

Business analysis

(Write 3–5 sentences from your own results. Questions to answer:) Were all vulnerabilities remediated? Authentication, rate limiting, logging and output filtering address three of the four findings in full; injection defense is the one that needs ongoing work because pattern matching alone blocked only 4 of 14 provided attacks in the reference run. Is the system ready for a SOC 2 audit? **[FILL IN]**. What additional controls might be needed? Candidates: semantic injection detection, HTTPS/TLS evidence, image and dependency scanning, an incident-response plan covering LLM scenarios, fail-closed rate limiting, and a documented data-residency position for GDPR.

Appendix — submission checklist

- Requirement 1: gateway config with JWT, IAM policies, 401 evidence, token tests.
- Requirement 2: three-tier config, 429 evidence, quota headers, burst config.
- Requirement 3: validation middleware, sanitization logic, your test results, security-event logging.
- Requirement 4: PII patterns, redaction rules, test results, redaction examples.
- Requirement 5: schema, ELK config, compliance matrix, sample queries, business analysis.
- Save as PDF; upload to Peer Review with a short description; review peers' work.